

AetherNet IoT & Autonomous Rover: Plataforma Distribuida de Domótica Modular, Telemetría Estadística y Robótica Móvil 100% FOSS

Andrés Felipe Martínez Henao
Tecnología en Desarrollo de Software – 5º semestre
Ingeniería de Sistemas y Computación – 6º semestre
Universidad Tecnológica de Pereira (UTP)
F.martinez5@utp.edu.co

Equipo AetherNet

Proyecto Integrador 2026-2 – 4 Sprints / 8 semanas

Repositorio: <https://github.com/Craos6518/AetherNet-IoT-Autonomous-Rover>

Abstract—AetherNet es una plataforma distribuida que integra domótica fija, control de acceso, robótica móvil y analítica estadística bajo una restricción estricta: 100% FOSS, sin dependencia de nubes propietarias (RNF-3.1). El sistema opera en tres capas sobre una misma LAN: (1) Edge – MEGA 2560 (teclado 4×4, servo MG90S, LED RGB, láser KY-008+LDR), UNO (rover con L298N, HC-SR04, 3×TCRT5000) y Gateway ESP32 (puente UART↔RF↔MQTT); (2) Coordinación – backend containerizado (FastAPI, PostgreSQL 16, Mosquitto 2.0) con 8 endpoints REST y bus aethernet/#; y (3) Interfaz – app Android nativa AetherControl (Kotlin, Compose, MVVM, Paho MQTT, joystick 20 Hz). La telemetría se suaviza con filtro EMA $S_t = \alpha Y_t + (1 - \alpha)S_{t-1}$, $\alpha = 0.2$ (HU-03) implementado tanto en Python (`stats/ema_filter.py`) como en firmware embebido, validado con 36.5k filas externas (Welch t + ANOVA). El proyecto se gestiona en Scrum con backlog MoSCoW (MOV/DEVOPS/LOW/PM/EST), 4 sprints y pipeline CI/CD de 6 jobs (`arduino-cli` 1.5.1, `ruff`/`mypy`/`pytest`, Trivy, Docker). Este artículo sintetiza visión, alcance, arquitectura, KPIs (<50 ms MQTT, <10 ms RF, >85% reducción de ruido, 0% falsos positivos láser) y estado al cierre de Sprint 3.

Index Terms—IoT FOSS, MQTT, ESP32, Arduino MEGA/UNO, nRF24L01, EMA, rover autónomo, Jetpack Compose, FastAPI, Scrum

I. INTRODUCCIÓN

A. Contexto académico

Proyecto Integrador que articula siete materias – Estadística (TS4D3), DevOps, Administración TS683, Programación Móvil TS6C3, Firmware, Hardware y Automatización Low-Code – desarrollado por estudiante de 5º semestre de Tecnología en Desarrollo de Software y 6º semestre de Ingeniería de Sistemas y Computación (UTP). El documento de referencia para agentes es `AGENTS.md`; el orden de lectura prescrito es:

- `prd.md` → `requirements.md` → `hardware-inventory.md`
- → `sprints.md` → `roadmap.md` → `backlog.md` → `docs/README.md`

B. Problema y visión

Como contingencia académica tras el sismo del 10 de agosto (7.4) que afectó parte del campus, AetherNet se plantea además como plataforma personal de estudio para demostrar que un ecosistema domótico+robótico robusto, tolerante a fallos y de baja latencia puede construirse exclusivamente con herramientas FOSS, eliminando vendor lock-in de AWS/Tuya Cloud. La visión (PRD §1) prioriza seguridad (control de acceso + trampa láser), inspección táctica (rover) y evidencia estadística reproducible.

C. Contribuciones

- (i) Arquitectura de 3 capas LAN-only con fail-safe RF;
- (ii) filtro EMA dual (Python↔firmware) con KPI verificable;
- (iii) app AetherControl con joystick de baja latencia y fallback Bluetooth SPP;
- (iv) metodología Scrum trazable RF↔HU↔backlog↔código.

II. ALCANCE Y REQUISITOS

A. In / Out / Cancelado / Futuro (PRD §4)

- **In (implementado):** app Android nativa, backend Docker, firmware C++ RF/UART/Wi-Fi, EMA $\alpha = 0.2$, LED RGB local, Telegram.
- **Out (fuera de alcance por diseño):** nubes de pago, iOS, visión artificial en rover.
- **Cancelado:** bombillo Tuya/`tuya-local` (ADR-001, 2026-09-01) por exigir cuenta propietaria y `local_key` (viola RNF-3.1). HU-02 se cumple solo con LED RGB + Telegram.
- **Futuro (no implementado en esta iteración):** flujo LowCode como referencia exportable (`automation/flows/intrusion_alert.json`) y dashboard Node-RED opcional.

B. Requisitos funcionales y no funcionales

RF-1.1 Dashboard Compose en tiempo real (MQTT/WS); RF-1.2 joystick X,Y→ESP32; RF-1.3 fallback Bluetooth SPP; RF-2.1 Gateway ESP32 como router MQTT↔UART↔RF; RF-2.2 MEGA teclado+servo; RF-2.3 láser KY-008; RF-3.1 rover nRF24L01; RF-3.2 evasión HC-SR04 + anti-caída TCRT5000; RF-3.3 fail-safe 500 ms (HU-04); RF-4.1 Telegram directo (HTTP api.telegram.org). RNF-1.1 docker-compose, RNF-1.2 CI `arduino-cli`, RNF-2.1 EMA, RNF-2.2 histórico PG, RNF-3.1 FOSS 100%.

C. Historias de usuario BDD

HU-01 (acceso PIN), HU-02 (intrusión láser→Telegram+LED rojo 3 s), HU-03 (EMA $\alpha = 0.2$ estabiliza HC-SR04), HU-04 (fail-stop RF). Criterios Dado/Cuando/Entonces son oracle de tests.

D. KPIs (PRD §5)

Latencia MQTT <50 ms, RF <10 ms, reducción ruido >85%, falsos positivos láser 0%, FOSS 100%. Cada KPI es verificable programáticamente (ej. `calculate_noise_reduction()` retorna `meets_kpi`).

III. ARQUITECTURA

A. Vista de 3 capas

Edge (MEGA/Uno/ESP32) → Coordinación (FastAPI+Mosquitto+PostgreSQL, topics `aethernet/rover/telemetry`, `access/event`, `seguridad/intrusion`, `rover/command`, `system/status`, ACL `aethernet/#`) → Interfaz (AetherControl + Telegram Bot). Única salida a Internet: Telegram Bot API (contingencia PRD §6: sin Internet, MEGA sigue operando Edge).

B. Protocolos y latencias

RF 2.4 GHz (nRF24L01, SPI, ch 76, 2 Mbps) <10 ms motor; UART 38400 bd MEGA↔Gateway; MQTT/WS <50 ms; Bluetooth SPP respaldo RF-1.3; HTTPS Telegram.

C. Flujo HU-02 (intrusión)

- 1) Láser KY-008 se interrumpe → MEGA detecta corte.
- 2) LED RGB local en rojo 3 s (sin red).
- 3) UART SECURITY: 38400 bd → Gateway.
- 4) Gateway publica `aethernet/seguridad/intrusion`.
- 5) Backend persiste en PG y dispara Telegram; App refleja alerta.

Pasos 2 y 5 son independientes (LED funciona sin MQTT).

D. Flujo EMA (HU-03)

- 1) Sensor HC-SR04 → firmware $S_t = 0.2Y_t + 0.8S_{t-1}$.
- 2) MQTT → PostgreSQL.
- 3) Offline: `stats/` Welch/ANOVA → notebook `EMA_Estadistica.ipynb`.

E. Decisiones y alternativas descartadas

MQTT vs polling (escalabilidad); RF vs Wi-Fi para rover (latencia); EMA vs Kalman (KPI con menor costo embebido); despliegue LAN-only.

IV. PLANIFICACIÓN SCRUM

A. Sprints

Sprint 1 (sem 1-2) Infra DEVOPS-01..05 + MOV-01 + EST-01; Sprint 2 (3-4) Domótica MEGA cerrojo/láser + MOV-02/03/04 + DEVOPS-06/07 + LOW-02 deuda; Sprint 3 (5-6) Rover L298N/HC-SR04/TCRT + MOV-05/06 joystick (Done 2026-09-11 [3ca12c9/e821700](https://github.com/3ca12c9/e821700)); Sprint 4 (7-8) LowCode Telegram directo, EST-02..07, MOV-08..10. Dependencias críticas: DEVOPS-01..05 bloquea todo; MOV-06←RF; EST-04←DEVOPS-12.

B. Backlog MoScOW y trazabilidad

5 áreas, IDs MOV/DEVOPS/LOW/PM/EST con columnas Prioridad/Sprint/Depende de/Origen. Convención commit: MOV-05: implementa joystick virtual. Cada Must no cerrado escala a `risk-register.md` y `gantt.md` (Mermaid).

C. Estado al 2026-09-11

Sprint 3 en curso, Sprint 2 cerrado documental (`docs/revision-sprint2-completa@ec5d70b`). Done: Docker 3 svc, Mosquitto `aethernet/#`, CI 6 jobs, RF validado HW `testing-rf-sprint1.md:32`, MEGA cerrojo+lun láser v2 `c7ce065`, app MVVM+MQTT+PIN+joystick 16 tests, datasets 36.5k. Pendiente: BT fallback, EMA KY-037, t-Student con latencias reales.

V. RESULTADOS POR SUBSISTEMA

A. Firmware/Hardware

MEGA: keypad ROW 22/24/26/28 COL 30/32/34/36, servo 9 (0/90°), LED 44/45/46 PWM ánodo común, láser TX 8 / LDR RX 7 INPUT 10k, UART 16/17 38400, auto-lock 5 s. Rover: UNO + L298N (ENA5/ENB11, IN1 6 IN2 7 IN3 8 IN4 9), TRIG2 ECHO3 + EMA $\alpha = 0.2$, TCRT A0 trasero/A1 izq/A2 der threshold 500, TT 6V 1:48 (calce 2026-09-11: -77% torque, 120 RPM, 0.35 m/s), fail-safe 500 ms + checksum, RF CE4/CSN10. Gateway ESP32 CE5/CSN15.

B. Backend y DevOps

`docker-compose.yml` (FastAPI 1.0.0-sprint1, Postgres 16, Mosquitto 2.0, net `aethernet-net`, `postgres_data`, `healthcheck pg_isready`, `depends_on healthy`). 8 endpoints (`routers/events.py:25`), 21 tests `pytest`. CI `ci.yml` 1.5.1: `backend-test` (ruff/mypy/pytest), `firmware-compile` (matriz ESP32/AVR), `docker-build` (`curl /health` 200), `stats-test`, `android-build` (condicional), `security-scan` Trivy SARIF.

C. App AetherControl

Kotlin + Compose + MVVM (StateFlow, DashboardUiState sellado), Room, DataStore, Retrofit, Paho 1.2.5 (tcp://1883 + ws://9001 fallback), ServiceLocator DI manual, Navigation. Done: LedStatusCard 64dp + poll 5 s, MqttManager SharedFlow, PinViewModel throttle 5/60 s + aethernet/access/command:70→UART CMD:ACCESS:40→ACCESS:hash 201, JoystickScreen Canvas 120dp + JoystickMapper deadband 60 + throttle 50 ms QoS0 + aethernet/rover/command:69 (250→249 99.6% TX).

D. Estadística

EMA prototipo ema_filter.py:15 + bench 531 muestras + water_turbidity_analysis.py Welch/ANOVA 36.5k (water_level_turbidity 31.5k CC BY-SA 4.0 + gesture 5k CC0, PNGs en docs/fritzing/).

E. Automatización LowCode

Telegram POST /api/security-events → sendMessage Markdown; flujo JSON (automation/flows/) solo como referencia exportable.

VI. DISCUSIÓN Y LIMITACIONES

La restricción FOSS forzó ADR-001 y deuda Node-RED, pero simplificó el deploy LAN y preservó la evaluabilidad. La concentración de EST en Sprint 4 se mitigó adelantando EST-01 y validación externa 36.5k. Limitaciones: T6 Prometheus/Grafana no implementado (logs Docker + aethernet/system/status heartbeat como MVP); presupuesto hardware real vs estimado aún informal; fotos de campo pendientes (checklist 12 MP). Riesgos críticos: LOW-01/02 cerrados, DEVOPS-01..05 como base fundacional, PM-05 (sprints.md desactualizado → agentes a ciegas).

VII. CONCLUSIONES

AetherNet demuestra viabilidad FOSS de un sistema IoT+robótica con trazabilidad completa requisito→código→test. Al cierre de Sprint 3 el núcleo domótico+rover es operable y medible contra KPIs; Sprint 4 cierra el lazo LowCode/estadístico y la demo E2E (HU-01→HU-04).

AGRADECIMIENTOS

A los docentes UTP de Estadística, Programación Móvil, Administración, DevOps y Automatización y LowCode por los lineamientos y la guía durante el proyecto. Firmware y Hardware se sustentan en formación previa.

REFERENCES

- [1] Equipo AetherNet, *PRD v1.0.0*, docs/prd.md, Ago 2026.
- [2] Equipo AetherNet, *Requirements Specification*, docs/requirements.md, Ago 2026.
- [3] Equipo AetherNet, *Arquitectura Técnica*, docs/architecture.md, Sep 2026.
- [4] Equipo AetherNet, *Hardware Inventory*, docs/hardware-inventory.md, Sep 2026.
- [5] Equipo AetherNet, *Planeación de Sprints*, docs/sprints.md, Sep 2026.
- [6] Equipo AetherNet, *Roadmap por Materia*, docs/roadmap.md, Sep 2026.
- [7] Equipo AetherNet, *Backlog MoSCoW*, docs/backlog.md, Sep 2026.
- [8] Equipo AetherNet, *ADR-001 Cancelación Tuya*, docs/adr/adr-001-cancelacion-tuya.md, Sep 2026.
- [9] Equipo AetherNet, *Diagrama Gantt PM-04*, docs/gantt.md, Sep 2026.
- [10] Equipo AetherNet, *Risk Register R-01..R-13*, docs/risk-register.md, Sep 2026.